SIMATS ENGINEERING
ASSESSMENT TOOL 6

COURSE NAME: Database Management Systems
COURSE CODE: CSA0506

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr. Hemavathi.R

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	**5**
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	**5**
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	**5**
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	**5**
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	**5**
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	**5**
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	**5**

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	**5**
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	**5**
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	**5**

Code of Conduct:

*I **G . J E E V E S H (1 9 2 5 1 1 4 4 0)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G.Jeevesh

1.Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

```
mysql>
mysql> SELECT * FROM Class;
+---------+-----------+---------------+
| ClassID | ClassName | TeacherName |
+---------+-----------+---------------+
|     101 | 10-A      | Radhika       |
|     102 | 10-B      | Suresh        |
|     103 | 9-A       | Anitha        |
+---------+-----------+---------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- STEP 7: DISPLAY STUDENT TABLE
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Student;
+-----+-------------+-------+-----------+
| SID | StudentName | Grade | ClassID |
+-----+-------------+-------+-----------+
|   1 | Rahul       | A     |       101 |
|   2 | Priya       | B     |       102 |
|   3 | Kiran       | A     |       101 |
|   4 | Anjali      | A+    |       103 |
+-----+-------------+-------+-----------+
4 rows in set (0.00 sec)

mysql>
mysql> -- STEP 8: LIST ALL STUDENTS WITH THEIR CLASS TEACHER AND GRADE
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT
    -> S.StudentName,
    -> C.ClassName,
    -> C.TeacherName,
    -> S.Grade
    -> FROM Student S
    -> INNER JOIN Class C
    -> ON S.ClassID = C.ClassID;
+-------------+-----------+---------------+-------+
| StudentName | ClassName | TeacherName | Grade |
+-------------+-----------+---------------+-------+
| Rahul       | 10-A      | Radhika       | A     |
| Kiran       | 10-A      | Radhika       | A     |
| Priya       | 10-B      | Suresh        | B     |
| Anjali      | 9-A       | Anitha        | A+    |
```

2. A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

```
mysql>
mysql> SELECT * FROM Appointment;
+---------------+-----------+----------+-----------------+
| AppointmentID | PatientID | DoctorID | AppointmentDate |
+---------------+-----------+----------+-----------------+
|          1001 |         1 |      101 | 2026-07-01      |
|          1002 |         2 |      101 | 2026-07-02      |
|          1003 |         3 |      101 | 2026-07-03      |
|          1004 |         4 |      101 | 2026-07-04      |
|          1005 |         5 |      101 | 2026-07-05      |
|          1006 |         1 |      101 | 2026-07-06      |
|          1007 |         2 |      101 | 2026-07-07      |
|          1008 |         3 |      101 | 2026-07-08      |
|          1009 |         4 |      101 | 2026-07-09      |
|          1010 |         5 |      101 | 2026-07-10      |
|          1011 |         1 |      101 | 2026-07-11      |
|          1012 |         2 |      102 | 2026-07-12      |
|          1013 |         3 |      103 | 2026-07-13      |
+---------------+-----------+----------+-----------------+
13 rows in set (0.00 sec)

mysql>
mysql> -- STEP 9: FIND DOCTORS WITH MORE THAN 10 APPOINTMENTS THIS MON
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT
    -> D.DoctorID,
    -> D.DoctorName,
    -> COUNT(A.AppointmentID) AS TotalAppointments
    -> FROM Doctor D
    -> JOIN Appointment A
    -> ON D.DoctorID = A.DoctorID
    -> WHERE MONTH(A.AppointmentDate) = 7
    -> AND YEAR(A.AppointmentDate) = 2026
    -> GROUP BY D.DoctorID, D.DoctorName
    -> HAVING COUNT(A.AppointmentID) > 10;
+----------+------------+--------------------+
| DoctorID | DoctorName | TotalAppointments |
+----------+------------+--------------------+
|      101 | Dr. Ravi   |                 11 |
+----------+------------+--------------------+
1 row in set (0.00 sec)
```

3.In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

```
+---------+-----------+----------+------------+
| OrderID | ProductID | Quantity | OrderDate  |
+---------+-----------+----------+------------+
|       1 |       101 |        2 | 2026-07-10 |
|       2 |       102 |        5 | 2026-07-12 |
|       3 |       104 |        1 | 2026-07-15 |
+---------+-----------+----------+------------+
3 rows in set (0.00 sec)

mysql>
mysql> -- STEP 8: FIND PRODUCTS THAT HAVE NEVER BEEN ORDERED (SUBQUERY)
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT ProductID, ProductName, Price
    -> FROM Product
    -> WHERE ProductID NOT IN
    -> (
    ->     SELECT ProductID
    ->     FROM Orders
    -> );
+-----------+-------------+---------+
| ProductID | ProductName | Price   |
+-----------+-------------+---------+
|       103 | Keyboard    | 1200.00 |
|       105 | Printer     | 8000.00 |
+-----------+-------------+---------+
2 rows in set (0.00 sec)

mysql>
mysql> -- STEP 9: ALTERNATIVE USING NOT EXISTS
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT P.ProductID, P.ProductName, P.Price
    -> FROM Product P
    -> WHERE NOT EXISTS
    -> (
    ->     SELECT *
    ->     FROM Orders O
    ->     WHERE O.ProductID = P.ProductID
    -> );
+-----------+-------------+---------+
| ProductID | ProductName | Price   |
+-----------+-------------+---------+
|       103 | Keyboard    | 1200.00 |
|       105 | Printer     | 8000.00 |
+-----------+-------------+---------+
```

4.For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

```
mysql>
mysql> SELECT S.Name
    -> FROM Student S
    -> JOIN Enrollment E1 ON S.SID=E1.SID
    -> JOIN Course C1 ON E1.CourseID=C1.CourseID
    -> JOIN Enrollment E2 ON S.SID=E2.SID
    -> JOIN Course C2 ON E2.CourseID=C2.CourseID
    -> WHERE C1.CourseName='DBMS'
    -> AND C2.CourseName='OS';
+--------+
| Name   |
+--------+
| Rahul  |
| Kiran  |
+--------+
2 rows in set (0.00 sec)
```

5. Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

```
mysql>
mysql> SELECT * FROM Employee;
+-------+---------+--------+-----------+
| EmpID | EmpName | DeptID | Salary    |
+-------+---------+--------+-----------+
|   101 | Rahul   |      1 | 55000.00  |
|   102 | Priya   |      1 | 70000.00  |
|   103 | Kiran   |      1 | 49500.00  |
|   104 | Anjali  |      2 | 44000.00  |
|   105 | Ravi    |      2 | 60000.00  |
|   106 | Sneha   |      3 | 55000.00  |
+-------+---------+--------+-----------+
6 rows in set (0.00 sec)
```

6. In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

```
    -> ON A.AccountNo = T.AccountNo;
+--------------+------------------+----------+-----------+-----------------+----------+
| CustomerName | BranchName       | City     | AccountNo | TransactionType | Amount   |
+--------------+------------------+----------+-----------+-----------------+----------+
| Rahul        | SBI Main Branch  | Hyderabad |     10001 | Deposit         | 10000.00 |
| Rahul        | SBI Main Branch  | Hyderabad |     10001 | Withdrawal      |  5000.00 |
| Kiran        | SBI Main Branch  | Hyderabad |     10003 | Withdrawal      |  3000.00 |
| Priya        | SBI City Branch  | Chennai  |     10002 | Deposit         | 20000.00 |
| Anjali       | SBI Central Branch | Bangalore |     10004 | Deposit         | 15000.00 |
+--------------+------------------+----------+-----------+-----------------+----------+
5 rows in set (0.00 sec)
```

7. A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

```
mysql>
mysql> SELECT * FROM Sales;
+--------+-----------+----------+------------+
| SaleID | ProductID | Quantity | SaleDate   |
+--------+-----------+----------+------------+
|      1 |       101 |        1 | 2026-07-30 |
|      2 |       102 |       10 | 2026-07-30 |
|      3 |       103 |        2 | 2026-07-30 |
|      4 |       104 |        1 | 2026-07-30 |
|      5 |       105 |       20 | 2026-07-30 |
|      6 |       106 |        1 | 2026-07-30 |
+--------+-----------+----------+------------+
6 rows in set (0.00 sec)

mysql>
mysql> -- STEP 8: TOTAL SALES PER CATEGORY EXCEEDING Rs.10000
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT
    -> P.Category,
    -> SUM(P.Price * S.Quantity) AS TotalSales
    -> FROM Product P
    -> JOIN Sales S
    -> ON P.ProductID = S.ProductID
    -> GROUP BY P.Category
    -> HAVING SUM(P.Price * S.Quantity) > 10000;
+-------------+------------+
| Category    | TotalSales |
+-------------+------------+
| Electronics |   67000.00 |
| Furniture   |   13000.00 |
+-------------+------------+
2 rows in set (0.00 sec)
```

8. For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

```
mysql>
mysql> SELECT * FROM Borrow;
+----------+----------+--------+------------+
| BorrowID | MemberID | BookID | BorrowDate |
+----------+----------+--------+------------+
|        1 |        1 |    101 | 2026-07-01 |
|        2 |        1 |    102 | 2026-07-02 |
|        3 |        1 |    103 | 2026-07-03 |
|        4 |        2 |    101 | 2026-07-04 |
|        5 |        2 |    102 | 2026-07-05 |
|        6 |        3 |    101 | 2026-07-06 |
|        7 |        3 |    102 | 2026-07-07 |
|        8 |        3 |    103 | 2026-07-08 |
+----------+----------+--------+------------+
8 rows in set (0.00 sec)

mysql>
mysql> -- STEP 9: MEMBERS WHO BORROWED BOOKS FROM ALL GENRES
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT M.MemberName
    -> FROM Member M
    -> WHERE NOT EXISTS
    -> (
    ->     SELECT DISTINCT Genre
    ->     FROM Book B
    ->     WHERE NOT EXISTS
    ->     (
    ->         SELECT *
    ->         FROM Borrow BR
    ->         JOIN Book BK
    ->         ON BR.BookID = BK.BookID
    ->         WHERE BR.MemberID = M.MemberID
    ->         AND BK.Genre = B.Genre
    ->     )
    -> );
+------------+
| MemberName |
+------------+
| Rahul      |
| Kiran      |
+------------+
2 rows in set (0.00 sec)
```

9. Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

```
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM Product;
+-----------+-------------+----------+----------+------------+
| ProductID | ProductName | Price    | Quantity | SupplierID |
+-----------+-------------+----------+----------+------------+
|         1 | Laptop      | 50000.00 |       20 |        101 |
|         2 | Mouse       |   500.00 |      100 |        101 |
|         3 | Keyboard    |  1200.00 |       50 |        102 |
+-----------+-------------+----------+----------+------------+
3 rows in set (0.00 sec)
```

10.Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

```
mysql>
mysql> SELECT * FROM MonthlyAttendance;
+-------+---------+------------+-------+-------------+
| EmpID | EmpName | Department | Month | DaysPresent |
+-------+---------+------------+-------+-------------+
|   101 | Rahul   | IT         | July  |          26 |
|   102 | Priya   | HR         | July  |          24 |
|   103 | Kiran   | Finance    | July  |          27 |
|   104 | Anjali  | IT         | July  |          25 |
+-------+---------+------------+-------+-------------+
4 rows in set (0.00 sec)

mysql>
mysql> -- STEP 11: CREATE VIEW FOR PERFORMANCE SUMMARY
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> CREATE VIEW PerformanceSummary AS
    -> SELECT
    -> E.EmpID,
    -> E.EmpName,
    -> E.Department,
    -> P.Rating
    -> FROM Employee E
    -> JOIN Performance P
    -> ON E.EmpID = P.EmpID;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- STEP 12: DISPLAY PERFORMANCE SUMMARY VIEW
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM PerformanceSummary;
+-------+---------+------------+--------+
| EmpID | EmpName | Department | Rating |
+-------+---------+------------+--------+
|   101 | Rahul   | IT         |    4.5 |
|   102 | Priya   | HR         |    4.0 |
|   103 | Kiran   | Finance    |    4.8 |
|   104 | Anjali  | IT         |    4.2 |
+-------+---------+------------+--------+
4 rows in set (0.00 sec)
```